WHAT DOES MY DESIGN DO?

1) Neural Network Model

I use a 1 hidden layer neural network with 8 neurons in it having a ReLu activation function connected to an output layer of 3 neurons with Softmax activation which operates on 4 parameters of a single input. Each input is classified by the model into one of the three classes- 0, 1 and 2. The input is taken from the Iris dataset from Kaggle. After the model is trained, the final weights, biases and even the test_data are saved to be used by the Verilog modules again.

2) Verilog Modules

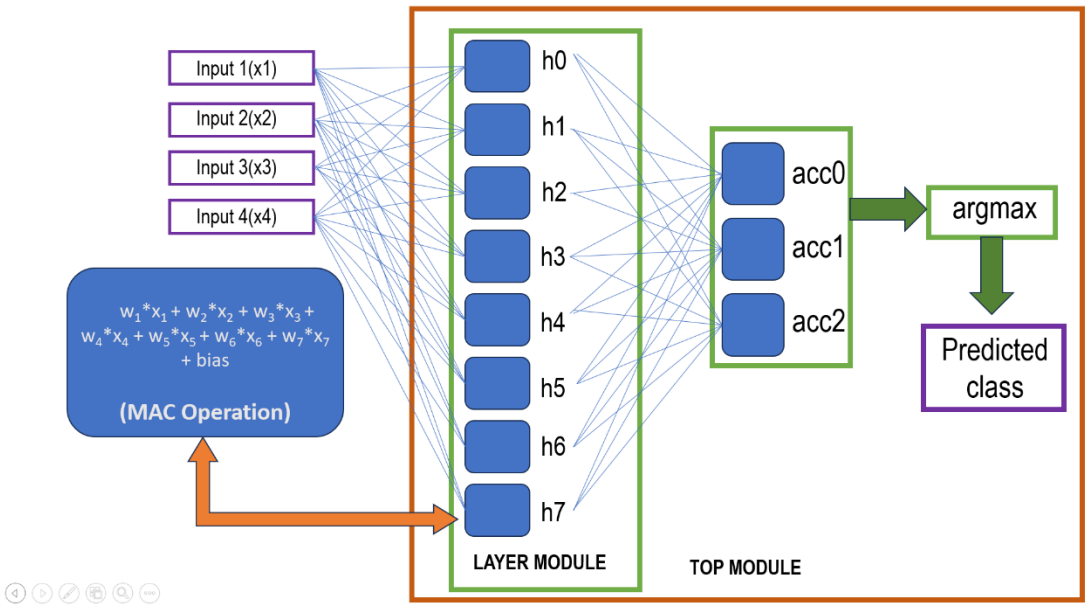
a) The building block is the ***Neuron module***. The basic functionality of this is to perform a MAC operation between the weights and biases and then finally present an output by passing this sum into a ReLu function.

b) A ***layer module*** is created which is nothing but 8 neurons created above working in parallel with the same 4 inputs. Here, only the weights and biases of each neuron are different but the working of each is same. As a result, 8 outputs from each activation of the 8 neurons are presented.

c) Lastly, the ***output layer*** is made which consists of 3 neurons whose inputs are the 8 outputs of the hidden layer. After the MAC operations, each neuron calculates a confidence number which are mapped to each prediction class {0, 1, 2}. Although a true SoftMax function is not explicitly implemented in hardware, the design approximates classification by selecting the neuron with the maximum output value (argmax), effectively determining the predicted class.

Finally, the trained weights and biases, are embedded into the hardware board, enabling the FPGA to perform real-time inference. Control signals such as start and done manage the inference process, and the final classification result is produced as a discrete output, demonstrating a complete pipeline from trained model parameters to hardware-based prediction.

BLOCK DIAGRAM



RESULTS

Finally, the *tb_nn_top.v* when run, gives accurate results for the output class. Here are some of the simulation runs-

```
================================
Prediction = 2
================================
acc0=   73658725 acc1=   43879265 acc2=  145188391
$finish called at time : 105 ns : File "C:/Users/IK/FPGA_MARK2/FPGA_MARK2.srcs/sim_1/new/tb_nn_top.v" Line 83
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_nn_top_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:06 ; elapsed = 00:00:08 . Memory (MB): peak = 1754.871 ; gain = 22.477
```

(Predicted class and all the three outputs of the output layer *acc0, acc1 & acc2* for the first 4 data points)

```
================================
Prediction = 1
================================
acc0=   21492856 acc1=   44294697 acc2=     344788
$finish called at time : 105 ns : File "C:/Users/IK/FPGA_MARK2/FPGA_MARK2.srcs/sim_1/new/tb_nn_top.v" Line 83
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_nn_top_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:04 ; elapsed = 00:00:07 . Memory (MB): peak = 1755.957 ; gain = 1.086
```

(Predicted class and all the three outputs of the output layer *acc0, acc1 & acc2* for the next 4 iterations of data points)

HARDEST PART I FOUND

Out of all, the most challenging part for me was debugging the neuron module. At first, the errors were mostly syntax based which from a minor research I could solve easily. But the main problem came when all the outputs (The ReLu output, acc and sum shown) were only shown as 'X'. I knew that some or the other data was not being read, because there was no error in any MAC operation. The ReLu function was also being applied correctly. Weights and biases were also loaded properly. Finally I learnt that the problem was with the dealing with clock signal. All the output was correctly calculated but the final output line was written in the same code block due to which the previous was being shown (which was null of course). So I had to write that line in the next clock cycle.